COMPARATIVE ANALYSIS OF TEXT CLASSIFICATION ALGORITHMS

Python NLP Laboratory — Step-by-step execution and explanation

Objective: Compare Naive Bayes, Support Vector Machine (SVM) and Centroid-Based Classification using the same text dataset and evaluate their performance.

1. Understanding the experiment

In this experiment, three classification algorithms are implemented using the same text dataset. The sample categories are Technology, Sports and Education. Each training document has a predefined category label. The models are trained on the same training documents and evaluated on the same testing documents to make the comparison fair.

Example documents include “Artificial Intelligence is growing” (Technology), “India won the cricket match” (Sports), and “Students attend college classes” (Education).

2. Understanding the three algorithms

Naive Bayes: A probability-based classifier. It calculates the probability that a document belongs to each category and selects the category with the highest probability.

Support Vector Machine (SVM): A classifier that identifies a decision boundary, or separating hyperplane, between categories.

Centroid-Based Classification: Calculates the average feature vector for each category and classifies a new document according to its similarity to the category centroids.

3. Purpose of comparative analysis

The experiment investigates which algorithm gives the highest accuracy, precision, recall and F1-score on the selected dataset, and whether the algorithms make the same predictions for new documents.

4. Common workflow

Common text dataset → training/testing split → TF-IDF feature extraction → Naive Bayes, SVM and Centroid-Based Classification → performance evaluation → comparative results.

All three algorithms use the same TF-IDF representation and the same training and testing split.

5. Install the required Python libraries

Open Command Prompt or the VS Code terminal and execute:

```
pip install scikit-learn pandas numpy
```

TfidfVectorizer converts text into numerical features; train_test_split divides the dataset; MultinomialNB implements Naive Bayes; SVC implements SVM; cosine_similarity compares documents with centroids; and the metrics functions evaluate performance.

6. Complete Python program

Create a Python file named **comparative_text_classification.py** and enter the following code:

```
# COMPARATIVE ANALYSIS OF TEXT CLASSIFICATION ALGORITHMS
# Algorithms: Naive Bayes, SVM and Centroid-Based Classification

# STEP 1: Import libraries
import numpy as np
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC

from sklearn.metrics.pairwise import cosine_similarity

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

# STEP 2: Create a common text dataset
documents = [
    # TECHNOLOGY - 10 DOCUMENTS
    "Artificial intelligence is transforming healthcare",
    "Machine learning is used in data science",
    "Python programming is useful for software development",
    "Deep learning is a branch of artificial intelligence",
    "Computer networks connect different devices",
    "Cloud computing provides online services",
    "Cybersecurity protects computer systems",
    "Robotics uses artificial intelligence",
    "Data mining discovers useful information",
    "Software engineering develops computer applications",

    # SPORTS - 10 DOCUMENTS
    "India won the cricket match",
    "Football is a popular sport",
    "The cricket team scored a century",
    "The football player scored a goal",
    "The team won the basketball tournament",
    "The tennis player won the championship",
    "The coach trained the cricket players",
    "The goalkeeper saved the penalty",
    "Athletes participated in the sports competition",
    "The player scored points in the match",

    # EDUCATION - 10 DOCUMENTS
    "Students attend classes in college",
    "Teachers explain programming concepts",
    "University conducts semester examinations",
    "Students prepare assignments and projects",
    "Online education provides learning opportunities",
    "The professor teaches computer science",
    "Students submit their laboratory records",
    "The college organizes technical workshops",
    "Teachers conduct practical examinations",
    "Students attend lectures in the classroom"
]

# STEP 3: Define category labels
labels = (
    ["Technology"] * 10 +
    ["Sports"] * 10 +
    ["Education"] * 10
)

# STEP 4: Divide data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    documents,
    labels,
    test_size=0.30,
    random_state=42,
    stratify=labels
)

print("Training documents:", len(X_train))
print("Testing documents:", len(X_test))

# STEP 5: Convert text into TF-IDF features
vectorizer = TfidfVectorizer(stop_words="english")
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

y_train = np.array(y_train)
y_test = np.array(y_test)

# STEP 6: MODEL 1 - NAIVE BAYES
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
nb_predictions = nb_model.predict(X_test_tfidf)

# STEP 7: MODEL 2 - SUPPORT VECTOR MACHINE
svm_model = SVC(kernel="linear")
svm_model.fit(X_train_tfidf, y_train)
svm_predictions = svm_model.predict(X_test_tfidf)

# STEP 8: MODEL 3 - CENTROID-BASED CLASSIFICATION
categories = np.unique(y_train)
centroids = []

for category in categories:
    category_documents = X_train_tfidf[y_train == category]
    centroid = category_documents.mean(axis=0)
    centroids.append(np.asarray(centroid).ravel())

centroids = np.array(centroids)
```

```
similarity = cosine_similarity(X_test_tfidf, centroids)
centroid_predictions = categories[
    np.argmax(similarity, axis=1)
]

# STEP 9: Common evaluation function
def evaluate_model(actual, predicted):
    accuracy = accuracy_score(actual, predicted)
    precision = precision_score(
        actual, predicted, average="weighted", zero_division=0
    )
    recall = recall_score(
        actual, predicted, average="weighted", zero_division=0
    )
    f1 = f1_score(
        actual, predicted, average="weighted", zero_division=0
    )
    return accuracy, precision, recall, f1

# STEP 10: Evaluate all three algorithms
nb_results = evaluate_model(y_test, nb_predictions)
svm_results = evaluate_model(y_test, svm_predictions)
centroid_results = evaluate_model(y_test, centroid_predictions)

# STEP 11: Display comparative performance
comparison = pd.DataFrame({
    "Algorithm": ["Naive Bayes", "SVM", "Centroid-Based"],
    "Accuracy": [nb_results[0], svm_results[0], centroid_results[0]],
    "Precision": [nb_results[1], svm_results[1], centroid_results[1]],
    "Recall": [nb_results[2], svm_results[2], centroid_results[2]],
    "F1-Score": [nb_results[3], svm_results[3], centroid_results[3]]
})

print("\n===== COMPARATIVE PERFORMANCE =====")
print(comparison.to_string(index=False))

# STEP 12: Display classification reports
models = {
    "Naive Bayes": nb_predictions,
    "SVM": svm_predictions,
    "Centroid-Based": centroid_predictions
}

for name, predictions in models.items():
    print("\n===== ", name, " =====")
    print(classification_report(
        y_test, predictions, zero_division=0
    ))

# STEP 13: Display confusion matrices
for name, predictions in models.items():
    print("\nCONFUSION MATRIX:", name)
    print(confusion_matrix(
        y_test,
        predictions,
        labels=["Education", "Sports", "Technology"]
    ))

# STEP 14: Identify the best-performing algorithm
best_index = comparison["F1-Score"].idxmax()
best_algorithm = comparison.loc[best_index, "Algorithm"]

print("\n===== BEST PERFORMING ALGORITHM =====")
print("Based on weighted F1-score:", best_algorithm)

# STEP 15: Classify new documents using all three algorithms
new_documents = [
    "Artificial intelligence helps computers learn",
    "India won the football tournament",
    "Students attend university classes"
]

new_tfidf = vectorizer.transform(new_documents)

nb_new = nb_model.predict(new_tfidf)
svm_new = svm_model.predict(new_tfidf)

centroid_similarity = cosine_similarity(new_tfidf, centroids)
centroid_new = categories[
    np.argmax(centroid_similarity, axis=1)
]

print("\n===== NEW DOCUMENT PREDICTIONS =====")
for i, document in enumerate(new_documents):
    print("\nDocument:", document)
    print("Naive Bayes:", nb_new[i])
    print("SVM:", svm_new[i])
    print("Centroid-Based:", centroid_new[i])
```

7. Explanation of the Python program

Step 1 — Import the libraries

NumPy and pandas support numerical operations and result presentation. TfidfVectorizer creates numerical text features. MultinomialNB implements Naive Bayes. SVC implements Support Vector Machine. cosine_similarity is used by the

centroid classifier. The metrics functions calculate classification performance.

Step 2 — Create the common dataset

The dataset contains 30 documents: 10 Technology, 10 Sports and 10 Education documents. The labels list provides the corresponding category for each document. All three algorithms use this same dataset.

Step 3 — Divide the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    documents, labels,
    test_size=0.30,
    random_state=42,
    stratify=labels
)
```

The split uses 70% for training (21 documents) and 30% for testing (9 documents). X_train and X_test contain the text documents; y_train and y_test contain their category labels. stratify=labels preserves the category proportions.

Step 4 — Convert text into TF-IDF features

```
vectorizer = TfidfVectorizer(stop_words="english")
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
```

fit_transform() learns the vocabulary and IDF weights from the training documents and converts them into vectors. transform() converts the test documents using the same learned vocabulary and weights. The same feature representation is supplied to all three algorithms.

Step 5 — Implement Naive Bayes

```
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
nb_predictions = nb_model.predict(X_test_tfidf)
```

MultinomialNB() creates the probability-based classifier. fit() trains it using the training vectors and labels. predict() predicts the categories of the testing documents.

Step 6 — Implement SVM

```
svm_model = SVC(kernel="linear")
svm_model.fit(X_train_tfidf, y_train)
svm_predictions = svm_model.predict(X_test_tfidf)
```

SVC() creates the SVM classifier. kernel='linear' uses a linear decision boundary. fit() trains the classifier and predict() classifies the test documents.

Step 7 — Implement centroid-based classification

```
categories = np.unique(y_train)
centroids = []

for category in categories:
    category_documents = X_train_tfidf[y_train == category]
    centroid = category_documents.mean(axis=0)
    centroids.append(np.asarray(centroid).ravel())

centroids = np.array(centroids)
```

The code identifies each category, selects its training documents, calculates the average TF-IDF vector and stores the resulting centroid. This is repeated for Technology, Sports and Education.

Step 8 — Calculate similarity and predict

```
similarity = cosine_similarity(X_test_tfidf, centroids)
centroid_predictions = categories[
    np.argmax(similarity, axis=1)
]
```

Cosine similarity compares each test document with all category centroids. np.argmax() selects the position of the highest similarity score, and the corresponding category becomes the prediction.

Step 9 — Evaluate the algorithms

The evaluate_model() function calculates accuracy, weighted precision, weighted recall and weighted F1-score. Using the same function for each model ensures that the same evaluation measures are applied consistently.

8. Understanding the evaluation metrics

Accuracy: Overall proportion of correct predictions.
Precision: Among documents predicted as a category, the proportion that actually belongs to that category.
Recall: Among documents that actually belong to a category, the proportion correctly identified.
F1-score: Harmonic mean of precision and recall.

The program uses weighted averages so that each category's contribution reflects its number of test documents.

9. Comparative performance output

The program displays a comparison table. Values are calculated during execution and should not be filled in with assumed results.

Algorithm	Accuracy	Precision	Recall	F1-score
Naive Bayes	Calculated	Calculated	Calculated	Calculated
SVM	Calculated	Calculated	Calculated	Calculated
Centroid-Based	Calculated	Calculated	Calculated	Calculated

Interpretation: higher accuracy indicates more correct predictions overall; higher precision indicates fewer false positives; higher recall indicates fewer actual documents are missed; and higher F1-score indicates a better balance between precision and recall.

10. Understanding the confusion matrix

A confusion matrix compares actual categories with predicted categories. Diagonal cells represent correct classifications, while off-diagonal cells represent misclassifications. The program prints a separate confusion matrix for each algorithm, using the same category order: Education, Sports and Technology.

11. Identify the best-performing algorithm

```
best_index = comparison["F1-Score"].idxmax()
best_algorithm = comparison.loc[
    best_index, "Algorithm"
]
```

The code finds the row with the highest weighted F1-score and displays the corresponding algorithm. This identifies the best performer on this particular test set; it does not establish that the algorithm is universally best.

12. Predict new documents

The program tests three new documents: “Artificial intelligence helps computers learn”, “India won the football tournament”, and “Students attend university classes”. Each document is classified by Naive Bayes, SVM and centroid-based classification so that their predictions can be compared.

13. How to execute the program

1. Open IDLE, VS Code, PyCharm or another Python environment.
2. Install the libraries using `pip install scikit-learn pandas numpy`.
3. Create a Python file named **comparative_text_classification.py**.
4. Paste the complete program and save it.
5. Open a terminal in the folder containing the file and run:

```
python comparative_text_classification.py
```

No external dataset is required because the sample documents are included in the program.

14. Important observations

After executing the program, students should answer:

1. Which algorithm achieved the highest accuracy?
2. Which algorithm achieved the highest precision?
3. Which algorithm achieved the highest recall?
4. Which algorithm achieved the highest F1-score?
5. Did all three algorithms classify the new documents identically?
6. Which algorithm would you select for this dataset, and why?

15. Student exercises

1. Increase the dataset from 30 to 60 documents.
2. Add a fourth category called Healthcare.
3. Replace TF-IDF with CountVectorizer and compare the results.
4. Change the SVM kernel from linear to RBF.
5. Compare results using different training/testing ratios.
6. Test the three algorithms using movie reviews (Positive and Negative).
7. Generate a bar chart comparing accuracy and F1-score.

16. Quick knowledge check

Question 1: Which algorithm is based on Bayes' theorem?
Answer: Naive Bayes.

Question 2: Which algorithm uses a separating hyperplane?
Answer: SVM.

Question 3: Which algorithm calculates the average feature vector for each category?
Answer: Centroid-Based Classification.

Expected laboratory outcome: Students should be able to implement Naive Bayes, SVM and centroid-based classification using a common text dataset, evaluate performance using accuracy, precision, recall and F1-score, and identify a suitable algorithm based on experimental results.

Teaching suggestion: First execute the three algorithms separately to explain their working principles. Then execute the combined comparative program and discuss the performance table and confusion matrices.

Research note: The 30-document dataset is designed for laboratory learning. A single small test split is not sufficient to establish that one algorithm is universally better. For research-level comparison, use a larger dataset, repeated stratified cross-validation and consistent hyperparameter tuning for all algorithms.